

1. SR-GAN

Model Architecture

- SR-GAN model is implemented using Pytorch on ImageNet datasets.
- The neural network architectures implemented here are for the generator and discriminator components of the SRGAN.
- The generator network consists of convolutional layers followed by residual blocks to enhance the resolution of input images. It includes a total of 16 residual blocks to capture intricate details.
- The discriminator network is designed to differentiate between high-resolution (HR) and generated high-resolution images. It employs convolutional layers with LeakyReLU activation to classify images as real or fake.
- Parameters such as the number of residual blocks in the generator (`num_residual_blocks`) and the slope of the LeakyReLU activation in the discriminator (`nn.LeakyReLU(0.2)`) are tuned to optimize model performance.
- Two main loss functions are defined in this network namely,
- Adversarial Loss (`criterion_GAN`): Utilizes Binary Cross-Entropy (BCE) loss. This loss function encourages the generator to produce images that are indistinguishable from real HR images by the discriminator.
- Content Loss (`criterion_content`): Employs Mean Squared Error (MSE) loss. This loss function ensures that the generated images maintain similar content to the ground truth HR images.
- Two Adam optimizers are utilized for training the generator and discriminator networks:
- Generator Optimizer (`optimizer_G`): Updates the parameters of the generator network to minimize the combined loss, including the adversarial and content losses.
- Discriminator Optimizer (`optimizer_D`): Updates the parameters of the discriminator network to better distinguish between real and generated images.
- The model is trained for 200 epochs to facilitate adversarial training, where the generator and discriminator networks compete against each other to improve image quality.
- A transformation pipeline (`transform`) is applied to each image pair before feeding it into the model.
- The evaluation of the super-resolution model involves processing test images and visualizing the outcomes, a crucial step in assessing model performance. To optimize memory usage and computation speed during this process, the procedure is encapsulated within a `'torch.no_grad()'` context, ensuring that no gradients are computed during the forward pass.

- The test image alongside the generated super-resolved image are exhibited side by side using matplotlib, facilitating a direct visual comparison. This setup enables an instant qualitative evaluation of the model's efficacy. Each image pair is depicted within a single figure containing two subplots, each appropriately labeled as 'Test Image' and 'Super-resolved Image'.

Experiments and Challenges:

1. Hyperparameters such as learning rates, content loss weight (λ_{content}), and the number of training epochs (`num_epochs`) were experimented based on empirical performance and convergence behavior.
2. Trained the loop for 200 epochs and it resulted in less clear images. If the training was done for 1000 epochs, the model would have produced high resolution images. This we could not do due to computation issues.

2. NLP

In this question, we experimented with a recurrent neural network (RNN) architecture, specifically a Long Short-Term Memory (LSTM) network, for the Jigsaw Toxic Comment Classification task. We aimed to build a model capable of identifying and classifying toxic comments in online conversations.

- **Preprocessing**

Text preprocessing was done on the training dataset 'comment_text' column. Text preprocessing techniques such as Special Character, Punctuation Removal, Lowercasing, tokenization using the word_tokenize, stopwords removal were applied on the training dataset. Finally, the processed tokens are joined back into a string using the join() method with a space separator, creating the 'clean_text' column.

- **Jigsaw Dataset Class**

A custom dataset class named 'Jigsaw' is created to preprocess the dataset and prepare it for training. This class takes a DataFrame (df) containing comments and their corresponding labels as input. It tokenizes the comments using a tokenizer function, pads the token sequences, and converts them to tensor format. Additionally, it creates tensors for the target labels. During initialization, we can specify whether the dataset is for testing or training purposes. If it's a test dataset, the target labels are initialized as zeros.

- **LSTM Model**

For the neural network architecture, we implemented an LSTM model using PyTorch. The 'LSTMModel' class consists of an embedding layer, an LSTM layer, and a fully connected layer. Here's a breakdown of each element:

Embedding Layer (`self.emb`): This layer converts input token indices into dense vectors of fixed size (`embedding\_dim`). It helps in capturing semantic similarities between words.

LSTM Layer (`self.lstm`): The LSTM layer processes the embedded input sequence and learns to capture long-range dependencies in the data. We used a single-layer LSTM with a hidden dimension of `hidden\_dim`.

Fully Connected Layer (`self.fc1`): The output of the LSTM layer is passed through a fully connected layer to produce the final output logits. The number of output units is set to 6, corresponding to the six target labels.

- **Loss Function and Optimization**

We used Binary Cross Entropy with Logits Loss (`nn.BCEWithLogitsLoss()`) as the loss function, which is suitable for multi-label classification tasks. The model was optimized using the Adam optimizer with a learning rate of 0.01 (`torch.optim.Adam()`).

- **Training Loop**

For each batch in the training data loader, the inputs and targets are extracted.

Forward pass: The inputs are fed into the model to obtain the outputs.

Sigmoid activation is applied to the outputs to obtain probabilities.

Loss calculation: Binary Cross Entropy with Logits Loss is computed between the predicted probabilities and the target labels.

Backward pass: Gradients are calculated and parameters are updated using the Adam optimizer.

The training loss for the batch is printed.

- Model is evaluated on the validation dataset and Predictions are made on the test dataset (test_dl) batch by batch.
- Probabilities from all batches are concatenated into a single tensor using `torch.cat(test_probs)`
- The tensor of probabilities is converted to a numpy array using `.detach().cpu().numpy()`. This ensures that the data is in CPU memory and detachs it from the computational graph.
- The numpy array of probabilities is assigned to the appropriate columns (target_cols) of the submission DataFrame (submission_df).
- Predicted Probabilities calculated for the submission df is saved as submission.csv and uploaded in canvas.
- Predicted labels for all target columns are calculated and stored as result_gt_pred.csv and uploaded to canvas
- Accuracy is calculated for each label and a combined accuracy is achieved as 91.7%.

Experiments and Challenges

Throughout the training process, we experimented with different hyperparameters, such as batch size, embedding dimension, hidden dimension, and learning rate, to optimize model performance. We also explored various preprocessing techniques, including tokenization and padding, to better represent the text data.

Issues:

Had issues with test labels as it ranged from -1 to 1 instead of 0,1. Removed the records having labels as -1 and considered only 0, 1 labels.

Encountered the issues while generating submissions file as the test file was not loaded properly in Google Colab. Fixed the issue and generated the file.